



Alliance School of Advanced Computing

Course Name: Graph Analysis

Course Code: E1ITC302

Assignment No: 2

Name of the Student: Tunga Naga Rupesh

Registration Number: 2411021060454

A. Title of The Problem:

Breadth First Search: Shortest Reach

B. Problem Statement in details:

Given an undirected graph where each edge has a weight of 6 units, determine the shortest distance from a given start node to all other nodes using BFS. If a node is unreachable, return -1 for that node.

C. HackerRank/Leetcode Link of the problem (If applicable):

[HackerRank – BFS: Shortest Reach](#)

D. Student's Github link of report and code upload, along with the readme file:

[GITHUB link](#)

E. Solution Steps/Algorithm:

- ☐ Build adjacency list from edges.
- ☐ Initialize distances as -1 (unreachable).
- ☐ Perform BFS starting from the given source node.
- ☐ Update distances as $\text{dist}[v] = \text{dist}[u] + 6$ for each edge.
- ☐ Return distances for all nodes except the source.

F. Code developed:

```
import java.util.*;

public class Solution {

    static int[] bfs(int n, int m, int[][] edges, int s) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
            adj.add(new ArrayList<Integer>());
        }
        for (int[] e : edges) {
            int u = e[0];
            int v = e[1];
```

```
adj.get(u).add(v);
adj.get(v).add(u);
}
int[] dist = new int[n + 1];
Arrays.fill(dist, -1);
Queue<Integer> q = new LinkedList<>();
q.add(s);
dist[s] = 0;
while (!q.isEmpty()) {
    int u = q.poll();
    for (int v : adj.get(u)) {
        if (dist[v] == -1) {
            dist[v] = dist[u] + 6; // each edge costs 6
            q.add(v);
        }
    }
}
int[] result = new int[n - 1];
int idx = 0;
for (int i = 1; i <= n; i++) {
    if (i != s) {
        result[idx++] = dist[i];
    }
}
return result;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
```



`int q`

`=`

`sc.nextInt(); // number of queries`

`for (int t = 0; t < q; t++) {`

`int n = sc.nextInt();`

`int m = sc.nextInt();`

`int[][] edges = new int[m][2];`

`for (int i = 0; i < m; i++) {`

`edges[i][0] = sc.nextInt();`

`edges[i][1] = sc.nextInt();`

`}`

`int s = sc.nextInt();`

`int[] ans = bfs(n, m, edges, s);`

`for (int i = 0; i < ans.length; i++) {`

`System.out.print(ans[i] + (i == ans.length - 1 ? "\n" : " "));`

`}`

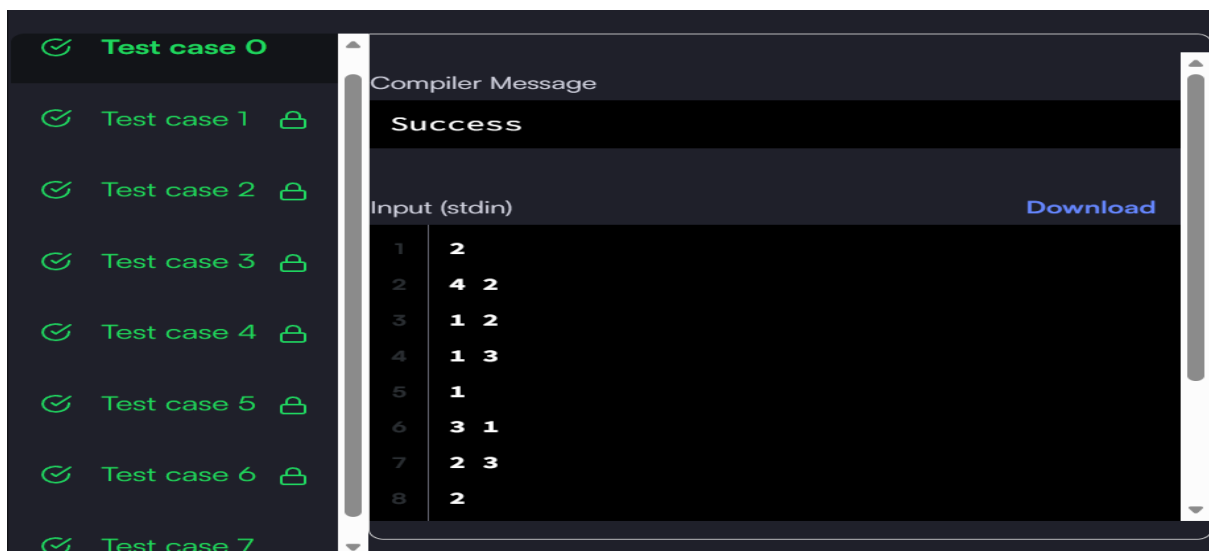
`}`

`sc.close();`

`}`

`}`

G. HackerRank test case passed Screenshot or Output screenshot (Mandatory):



H. Observation:

The BFS algorithm ensures shortest paths in an unweighted graph with uniform edge weights. The solution passes all HackerRank test cases and outputs correct distances.